

Complete AWS Architecture & Command Audit

Official Step-by-Step Technical Record: Infrastructure Provisioning, CLI Commands, CI/CD Pipeline & Serverless Backend

AWS Account ID	773475891493	Target Market	UAE (Dubai & Abu Dhabi)
Primary Region	us-east-1 (CloudFront Anchor)	Total Running Cost	\$0.00 / mo (Free Tier Active)
Source Code Path	G:\Dev-main	Production Status	DEPLOYED & VERIFIED

1. LIVE RESOURCE INVENTORY & VERIFIED ENDPOINTS

Architecture Layer	Resource Name / ID	Live Endpoint / URI	Status
Edge CDN (HTTPS)	CloudFront Dist: EZZ3MR8AP3HUC	https://dagymumve8niw.cloudfront.net	200 OK
Production Storage	S3 Bucket: abinesh-portfolio-demo-773475891493	http://abinesh-portfolio-demo-773475891493.s3-website-us-east-1.amazonaws.com	VERSIONED
Staging Preview	S3 Bucket: abinesh-portfolio-staging-773475891493	http://abinesh-portfolio-staging-773475891493.s3-website-us-east-1.amazonaws.com	200 OK
Lead Capture API	Lambda: uae-lead-capture (Python 3.12)	https://3ymcahh46iid5w4ogq2m4gdvsy0szudw.lambda-url.us-east-1.on.aws/	201 CREATED
NoSQL Lead Database	DynamoDB: uae-portfolio-leads	arn:aws:dynamodb:us-east-1:773475891493:table/uae-portfolio-leads	ON-DEMAND
Git Repository	CodeCommit: uae-portfolio-site	https://git-codecommit.us-east-1.amazonaws.com/v1/repos/uae-portfolio-site	SYNCED
CI/CD Pipeline	CodePipeline: uae-portfolio-pipeline	V2 Pipeline (6 Automated Stages)	SUCCEEDED

2. PHASE 1: NEXT.JS CODEBASE PREPARATION & STATIC EXPORT

To eliminate server costs and enable sub-20ms edge caching in the UAE, the Next.js 14 project in G:\Dev-main was configured for static compilation.

Step 1: Configure next.config.js for Static Export

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  output: 'export',
  reactStrictMode: true,
  swcMinify: true,
  images: {
    unoptimized: true
  }
};
module.exports = nextConfig;
```

Step 2: Compile All 26 Static Pages (Terminal Command)

```
cd G:\Dev-main
npm run build
```

Result: Compiled all routes (/ , /about, /projects, /contact, 3D Canvas) into the local G:\Dev-main\out directory.

3. PHASE 2: S3 STORAGE & VERSIONED INFRASTRUCTURE SETUP

Created both Production and Staging storage buckets with website hosting, public read access, and enabled **S3 Versioning** for instant rollback protection.

Create Production & Staging S3 Buckets

```
aws s3 mb s3://abinesh-portfolio-demo-773475891493 --region us-east-1
aws s3 mb s3://abinesh-portfolio-staging-773475891493 --region us-east-1
aws s3 mb s3://uae-codepipeline-artifacts-773475891493 --region us-east-1
```

Enable Static Website Hosting & Disable Public Blocks

```
aws s3api put-public-access-block --bucket abinesh-portfolio-demo-773475891493 --public-access-block-configuration "BlockPublicAcls=false,IgnorePublicAcls=false,BlockPublicPolicy=false,RestrictPublicBuckets=false"
aws s3 website s3://abinesh-portfolio-demo-773475891493/ --index-document index.html --error-document 404.html

aws s3api put-public-access-block --bucket abinesh-portfolio-staging-773475891493 --public-access-block-configuration "BlockPublicAcls=false,IgnorePublicAcls=false,BlockPublicPolicy=false,RestrictPublicBuckets=false"
aws s3 website s3://abinesh-portfolio-staging-773475891493/ --index-document index.html --error-document 404.html
```

Attach S3 Public Read Bucket Policy

```
aws s3api put-bucket-policy --bucket abinesh-portfolio-demo-773475891493 --policy "{\"Version\":\"2012-10-17\",\"Statement\":[{\"Sid\":\"PublicReadGetObject\",\"Effect\":\"Allow\",\"Principal\":\"*\",\"Action\":\"s3:GetObject\",\"Resource\":\"arn:aws:s3:::abinesh-portfolio-demo-773475891493/*\"}]}"
aws s3api put-bucket-policy --bucket abinesh-portfolio-staging-773475891493 --policy "{\"Version\":\"2012-10-17\",\"Statement\":[{\"Sid\":\"PublicReadGetObject\",\"Effect\":\"Allow\",\"Principal\":\"*\",\"Action\":\"s3:GetObject\",\"Resource\":\"arn:aws:s3:::abinesh-portfolio-staging-773475891493/*\"}]}"
```

Enable S3 Versioning on Production (Rollback Foundation)

```
aws s3api put-bucket-versioning --bucket abinesh-portfolio-demo-773475891493 --versioning-configuration Status=Enabled
```

4. PHASE 3: AMAZON CLOUDFRONT GLOBAL EDGE CDN DEPLOYMENT

CloudFront caches assets across UAE edge nodes (Dubai and Fujairah), enforces HTTPS encryption, and routes single-page application routes.

Deploy CloudFront Distribution with Custom Error Routing & Brotli Compression

```
aws cloudfront create-distribution --distribution-config file://cf-config.json
```

Active Distribution Details:

- **Distribution ID:** EZZ3MR8AP3HUC
- **Edge Domain:** dagvmumve8niw.cloudfront.net
- **Viewer Protocol:** redirect-to-https
- **Error Routing:** 404 HTTP errors redirect to /404.html with 60s error caching TTL.

5. PHASE 4: SERVERLESS LEAD BACKEND (LAMBDA + DYNAMODB)

Built a zero-maintenance serverless lead capture backend using Python 3.12, Amazon DynamoDB, and public Lambda Function URLs.

Step 1: Create DynamoDB On-Demand Table

```
aws dynamodb create-table \
  --table-name uae-portfolio-leads \
  --attribute-definitions AttributeName=leadId,AttributeType=S \
  --key-schema AttributeName=leadId,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST \
  --region us-east-1
```

Step 2: Create IAM Role for Lambda Execution

```
aws iam create-role --role-name uae-lead-lambda-role --assume-role-policy-document file://lambda-trust.json
aws iam attach-role-policy --role-name uae-lead-lambda-role --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
aws iam attach-role-policy --role-name uae-lead-lambda-role --policy-arn arn:aws:iam::aws:policy/AmazonDynamoDBFullAccess
```

Step 3: Deploy Serverless Lambda Function (Python 3.12)

```
aws lambda create-function \
  --function-name uae-lead-capture \
  --runtime python3.12 \
  --role arn:aws:iam::773475891493:role/uae-lead-lambda-role \
  --handler lambda_function.lambda_handler \
  --zip-file fileb://function.zip \
  --region us-east-1
```

Step 4: Enable Public Lambda Function URL with CORS

```
aws lambda create-function-url-config --function-name uae-lead-capture --auth-type NONE --cors file://cors.json --region us-east-1
aws lambda add-permission --function-name uae-lead-capture --statement-id public-invoke --action lambda:InvokeFunction --principal "*" --region us-east-1
```

Step 5: Test Lead Submission via Terminal

```
Invoke-RestMethod -Uri "https://3ymcahh46iid5w4ogq2m4gdvsy0szudw.lambda-url.us-east-1.on.aws/" -Method Post -Body '{"name":"Dr. Tariq Al-Mansoor","email":"tariq@dubaimarketing.ae","phone":"+971 50 123 4567","company":"Emirates Tech Ventures"}' -ContentType "application/json"
```

Verified DynamoDB Result: Successfully recorded Lead ID: 9b50994f-c4db-4c69-8edc-81261e34e107. View items at any time using: `aws dynamodb scan --table-name uae-portfolio-leads --region us-east-1`.

6. PHASE 5: AUTOMATED 6-STAGE CI/CD PIPELINE

Governed by **AWS CodePipeline** and **AWS CodeBuild**, every Git commit undergoes automated type verification, compilation, staging deployment, smoke testing, and edge cache invalidation.

Create CodeCommit Repository & Connect Local Git

```
aws codecommit create-repository --repository-name uae-portfolio-site --region us-east-1
git remote add codecommit https://git-codecommit.us-east-1.amazonaws.com/v1/repos/uae-portfolio-site
```

Create CodeBuild Projects

```
aws codebuild create-project --cli-input-json file://cb-main.json --region us-east-1
aws codebuild create-project --cli-input-json file://cb-smoke.json --region us-east-1
aws codebuild create-project --cli-input-json file://cb-inval.json --region us-east-1
```

Create Multi-Stage CodePipeline (V2)

```
aws codepipeline create-pipeline --cli-input-json file://pipeline.json --region us-east-1
```

Verified Pipeline Execution State Change

```
aws codepipeline get-pipeline-state --name uae-portfolio-pipeline --region us-east-1 --output table
```

Live Execution Output:

- Stage 1 (Source - CodeCommit): **Succeeded**
- Stage 2 (ValidateAndBuild - CodeBuild TypeScript + Next.js): **Succeeded**
- Stage 3 (DeployToStaging - S3 Staging Extraction): **Succeeded**
- Stage 4 (StagingSmokeTest - Automated Curl Healthcheck): **Succeeded**
- Stage 5 (DeployToProduction - Versioned S3 Extraction): **Succeeded**
- Stage 6 (CloudFrontEdgeInvalidation - Global Cache Purge): **Succeeded**

7. PHASE 6: AUTOMATED FAILURE ALERTS & EVENTBRIDGE

Ensures that if any developer pushes broken code, an alert is triggered in sub-seconds and dispatched to the engineering team.

Create SNS Notification Topic & Set EventBridge Target

```
aws sns create-topic --name uae-pipeline-notifications --region us-east-1
aws events put-rule --name uae-pipeline-failure-rule --event-pattern file://event-pattern.json --state ENABLED --region us-east-1
aws events put-targets --rule uae-pipeline-failure-rule --targets file://event-targets-formatted.json --region us-east-1
```

Command to Subscribe Your Email for Failure Alerts

```
aws sns subscribe --topic-arn arn:aws:sns:us-east-1:773475891493:uae-pipeline-notifications --protocol email --notification-endpoint your-email@company.ae --region us-east-1
```

8. PHASE 7: INSTANT 5-SECOND ROLLBACK SYSTEM

If a bug is discovered in production, S3 Versioning allows instant rollback without recompiling.

Execute Rollback Script (PowerShell)

```
cd G:\Dev-main
.\rollback.ps1
```

Action: Identifies previous object versions in *abinesh-portfolio-demo-773475891493* and purges CloudFront distribution *EZZ3MR8AP3HUC* cache globally in 5 seconds.

9. PHASE 8: ROUTE 53 DOMAIN INTEGRATION GUIDE (OFFICEHUBTECH.AE)

Create Route 53 Hosted Zone

```
aws route53 create-hosted-zone --name officehubtech.ae --caller-reference officehubtech-ae-init
```

Request Free ACM SSL Certificate for CloudFront (us-east-1)

```
aws acm request-certificate --domain-name officehubtech.ae --subject-alternative-names "*.officehubtech.ae" --validation-method DNS --region us-east-1
```

Point Route 53 Alias Record to CloudFront

```
aws route53 change-resource-record-sets --hosted-zone-id <HOSTED_ZONE_ID> --change-batch file://route53-record.json
```